



Universidad Don Bosco

Campus virtual

Facultad de Ingeniería

Foro de Discusión 2

Desarrollo de una aplicación de control de gastos personales con Firebase y Kotlin en

Android Studio

Asignatura: Desarrollo de Software para Móviles

Código: DSM104

Docente:

Ing. Alexander Alberto Siguenza Campos

Integrantes:

Ronald Vladimir Urias Orellana — RO230537

Karen Banessa Ochoa García — OG241850

Rolando Giovanni Guevara Martir — GM202436

Daniela Fernanda Tamayo Jovel — TJ231470

San Salvador, El Salvador

10 de mayo de 2026

Índice

1. Introducción

2. Investigación sobre Firebase Authentication

2.1. Concepto de Firebase Authentication

2.2. Tipos de autenticación disponibles en Firebase

2.3. Autenticación por correo electrónico y contraseña

2.4. Autenticación con Google

2.5. Autenticación por teléfono

2.6. Otros métodos de autenticación disponibles

2.7. Ventajas, limitaciones y consideraciones de seguridad

3. Desarrollo técnico de la implementación

3.1. Creación del proyecto en Android Studio

3.2. Ejecución inicial de la aplicación

3.3. Creación del proyecto en Firebase Console

3.4. Registro de la aplicación Android en Firebase

3.5. Integración del archivo google-services.json

3.6. Configuración del SDK de Firebase en Gradle

3.7. Activación de Firebase Authentication

3.8. Configuración de Firebase Firestore

3.9. Reglas de seguridad de Firestore

4. Desarrollo de la aplicación de control de gastos

4.1. Diseño general de la aplicación

4.2. Registro e inicio de sesión con correo electrónico

4.3. Registro de gastos

4.4. Almacenamiento de gastos en Firestore

4.5. Visualización del historial de gastos

4.6. Cálculo del total mensual

4.7. Validaciones y manejo de errores

5. Capturas de pantalla del proyecto en ejecución

6. Pruebas realizadas

7. Conclusiones personales del grupo

8. Referencias

9. Anexos

1. Introducción

El control de gastos personales es una práctica importante para organizar el uso del dinero y conocer con mayor claridad en qué se realizan los principales consumos. Muchas personas hacen gastos diarios sin llevar un registro ordenado, lo que dificulta identificar cuánto dinero han utilizado durante el mes y cuáles son las categorías donde más gastan.

El presente proyecto consiste en el desarrollo de una aplicación móvil para Android orientada al control de gastos personales. La aplicación permite que los usuarios inicien sesión, registren gastos con nombre, monto, categoría y fecha, visualicen un historial de gastos y consulten el total mensual acumulado.

Para el desarrollo de la aplicación se utilizó Android Studio como entorno de trabajo, Kotlin como lenguaje de programación y Jetpack Compose para la construcción de la interfaz gráfica. Además, se integró Firebase Authentication para gestionar el inicio de sesión de los usuarios y Firebase Firestore para almacenar los gastos en la nube (Android Developers, s. f.-a; Android Developers, s. f.-b; Firebase, s. f.-b; Firebase, s. f.-d).

La aplicación asocia cada gasto al usuario autenticado, evitando que la información de diferentes usuarios se mezcle. De esta manera, se construye una solución funcional, clara y organizada que aplica conceptos de desarrollo móvil, autenticación, almacenamiento en la nube y buenas prácticas de programación.

El objetivo principal del proyecto es crear una aplicación móvil que permita gestionar gastos personales mediante autenticación con Firebase, registro de información financiera básica y visualización de un resumen mensual, cumpliendo con los requisitos técnicos establecidos para la actividad.

2. Investigación sobre Firebase Authentication

2.1. Concepto de Firebase Authentication

Firebase Authentication es un servicio de Firebase que permite implementar sistemas de autenticación en aplicaciones móviles y web. Su objetivo principal es facilitar el registro, inicio de sesión y administración de usuarios mediante diferentes métodos de acceso (Firebase, s. f.-d).

Este servicio evita que el desarrollador tenga que crear desde cero toda la lógica de autenticación, ya que proporciona herramientas para identificar usuarios de forma segura. Además, puede integrarse con otros servicios de Firebase, como Firestore, permitiendo asociar la información almacenada con el usuario autenticado.

En el contexto de este proyecto, Firebase Authentication se utiliza para permitir que cada usuario pueda ingresar a la aplicación y administrar únicamente sus propios gastos personales.

2.2. Tipos de autenticación disponibles en Firebase

Firebase Authentication permite trabajar con diferentes métodos de autenticación, incluyendo proveedores externos, autenticación anónima y autenticación personalizada (Firebase, s. f.-d). Entre los principales métodos considerados para el proyecto se encuentran:

- Autenticación por correo electrónico y contraseña.
- Autenticación con Google.
- Autenticación por teléfono.
- Autenticación con proveedores externos como Apple, Facebook, GitHub,

Microsoft, Yahoo y X.

- Autenticación anónima.
- Autenticación personalizada mediante tokens.

Cada método tiene características diferentes y puede utilizarse según el tipo de aplicación, el nivel de seguridad requerido y la comodidad que se desea ofrecer al usuario.

2.3. Autenticación por correo electrónico y contraseña

La autenticación por correo electrónico y contraseña permite que el usuario cree una cuenta utilizando su correo personal y una contraseña. Es uno de los métodos más comunes en aplicaciones móviles, ya que resulta familiar para la mayoría de usuarios.

Este método permite registrar usuarios, iniciar sesión, cerrar sesión y validar credenciales. Para utilizarlo correctamente, la aplicación debe verificar que el correo no esté vacío, que tenga un formato válido y que la contraseña cumpla una longitud mínima.

Ventajas:

- Es fácil de entender para el usuario.
- No depende de redes sociales ni de proveedores externos adicionales.
- Permite que cada usuario tenga una cuenta propia dentro de la aplicación.
- Es adecuado para aplicaciones académicas, personales y empresariales.

Limitaciones:

- El usuario debe recordar una contraseña.
- Se deben manejar errores como correo inválido, contraseña incorrecta o usuario no registrado.
- Requiere aplicar validaciones para evitar registros incorrectos.

Caso de uso:

Este método es adecuado para una aplicación de control de gastos personales, porque permite identificar a cada usuario y asociar sus gastos a una cuenta específica.

2.4. Autenticación con Google

La autenticación con Google permite que el usuario inicie sesión utilizando una cuenta de Google. Este método mejora la experiencia de uso porque evita que el usuario tenga que crear una nueva contraseña para la aplicación (Firebase, s. f.-a).

Cuando el usuario selecciona la opción de iniciar sesión con Google, Firebase valida la cuenta y permite que la aplicación identifique al usuario autenticado. Este método es especialmente útil en dispositivos Android, donde muchas personas ya tienen una cuenta de Google activa.

Ventajas:

- Permite un inicio de sesión rápido.
- Reduce el tiempo de registro.
- Disminuye la necesidad de recordar otra contraseña.
- Aprovecha los mecanismos de seguridad de Google.

Limitaciones:

- Requiere configurar correctamente el proveedor Google en Firebase.
- Puede requerir la configuración del certificado SHA-1 o SHA-256 en el proyecto.
- Depende de que el usuario tenga o utilice una cuenta de Google.

Caso de uso:

Este método es recomendable para aplicaciones móviles que buscan ofrecer un acceso rápido y cómodo. En este proyecto se incluye porque la actividad solicita implementar inicio de sesión con Google.

2.5. Autenticación por teléfono

La autenticación por teléfono permite verificar al usuario mediante un número telefónico. Normalmente, el usuario ingresa su número y recibe un código de verificación por SMS, el cual debe introducir en la aplicación para completar el inicio de sesión.

Ventajas:

- No requiere correo electrónico ni contraseña.
- Puede ser útil cuando el número telefónico es el dato principal del usuario.
- Permite verificar que el usuario tenga acceso al teléfono registrado.

Limitaciones:

- Depende del envío de mensajes SMS.
- Puede presentar restricciones según el país o la compañía telefónica.
- Puede generar costos o límites de uso.
- Requiere medidas para evitar abuso en el envío de códigos.

Caso de uso:

La autenticación por teléfono es común en aplicaciones de servicios, entregas, transporte, banca móvil o sistemas donde el número telefónico funciona como identificador principal del usuario.

2.6. Otros métodos de autenticación disponibles

Además de correo electrónico, Google y teléfono, Firebase Authentication permite utilizar otros proveedores de identidad. Entre ellos se encuentran Apple, Facebook, GitHub, Microsoft, Yahoo y X. También permite autenticación anónima y autenticación personalizada (Firebase, s. f.-d).

La autenticación anónima puede utilizarse cuando se desea que el usuario pruebe una aplicación sin crear una cuenta. Sin embargo, en una aplicación de control de gastos personales no es la opción más adecuada, ya que los datos deben estar claramente asociados a un usuario.

La autenticación personalizada se utiliza cuando una aplicación ya tiene su propio sistema de usuarios y necesita conectarlo con Firebase mediante tokens. Este método es más avanzado y se utiliza en sistemas con necesidades específicas.

2.7. Ventajas, limitaciones y consideraciones de seguridad

Firebase Authentication ofrece varias ventajas para el desarrollo de aplicaciones móviles. Permite implementar autenticación sin construir todo el sistema desde cero, se integra fácilmente con otros servicios de Firebase y ofrece distintos métodos de acceso para adaptarse a diferentes usuarios.

Sin embargo, también requiere una configuración correcta. En Android, el nombre del paquete debe coincidir con el registrado en Firebase. Además, para algunos proveedores, como Google, puede ser necesario configurar certificados de seguridad como SHA-1 o SHA-256.

En cuanto a seguridad, la aplicación debe validar los datos ingresados antes de enviarlos a Firebase. El correo no debe estar vacío, debe tener formato válido y la contraseña debe cumplir una longitud mínima. También se deben mostrar mensajes claros cuando exista un error de autenticación.

Otra consideración importante es proteger la información almacenada en Firestore. Cada usuario debe acceder únicamente a sus propios gastos. Para lograrlo, se deben utilizar reglas de seguridad que verifiquen que el usuario autenticado coincida con el identificador del documento o ruta que intenta leer o modificar (Firebase, s. f.-c; Firebase, s. f.-e).

3. Desarrollo técnico de la implementación

3.1. Creación del proyecto en Android Studio

Para iniciar el desarrollo de la aplicación, se creó un nuevo proyecto en Android Studio con el nombre `ControlGastosPersonales`. El lenguaje seleccionado fue Kotlin, cumpliendo con el requisito técnico establecido para la actividad. Para la interfaz gráfica se utilizó Jetpack Compose, herramienta oficial para construir interfaces declarativas en Android (Android Developers, s. f.-b; Kotlin, s. f.).

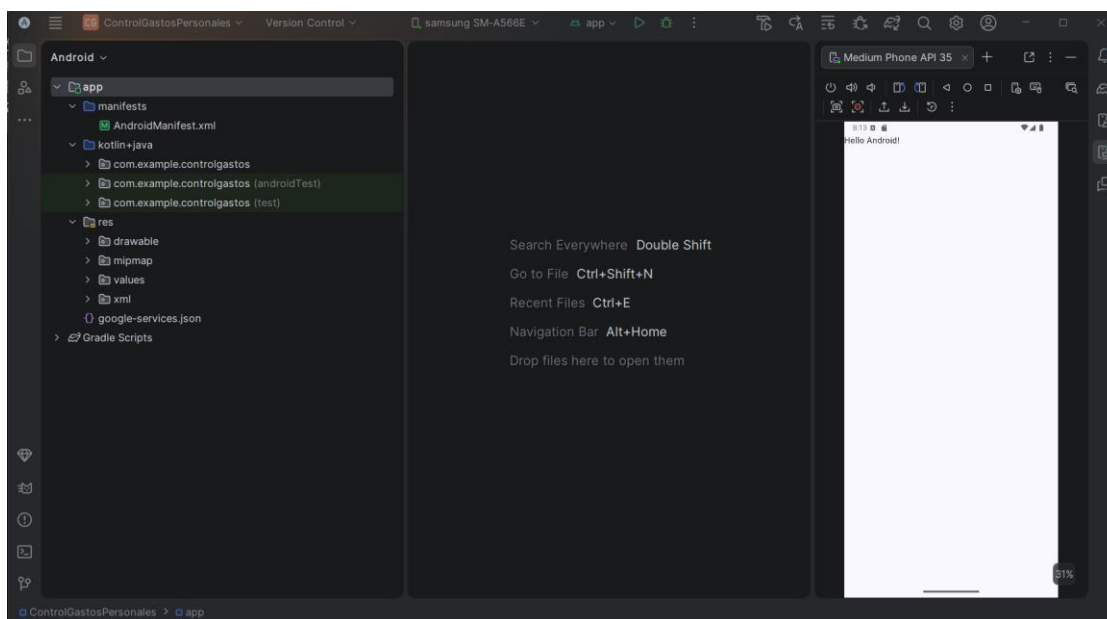
Durante la creación del proyecto se configuró el nombre del paquete como:

`com.example.controlgastos`

Este nombre de paquete es importante porque debe coincidir con el paquete registrado posteriormente en Firebase.

Figura 1

Proyecto ControlGastosPersonales creado en Android Studio.



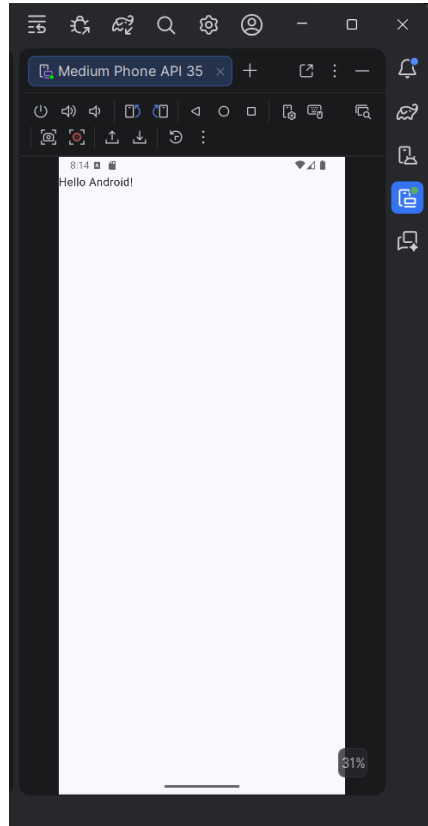
Nota. Se muestra el proyecto Android creado en Android Studio con Kotlin y Jetpack Compose.

3.2. Ejecución inicial de la aplicación

Antes de integrar Firebase, se ejecutó la aplicación base en el emulador de Android Studio para comprobar que el proyecto compilara y funcionara correctamente. La ejecución inicial permitió verificar que el entorno de desarrollo estaba configurado de forma adecuada.

Figura 2

Ejecución inicial de la aplicación en el emulador Android.



Nota. Se observa la aplicación base ejecutándose correctamente en el emulador antes de integrar Firebase.

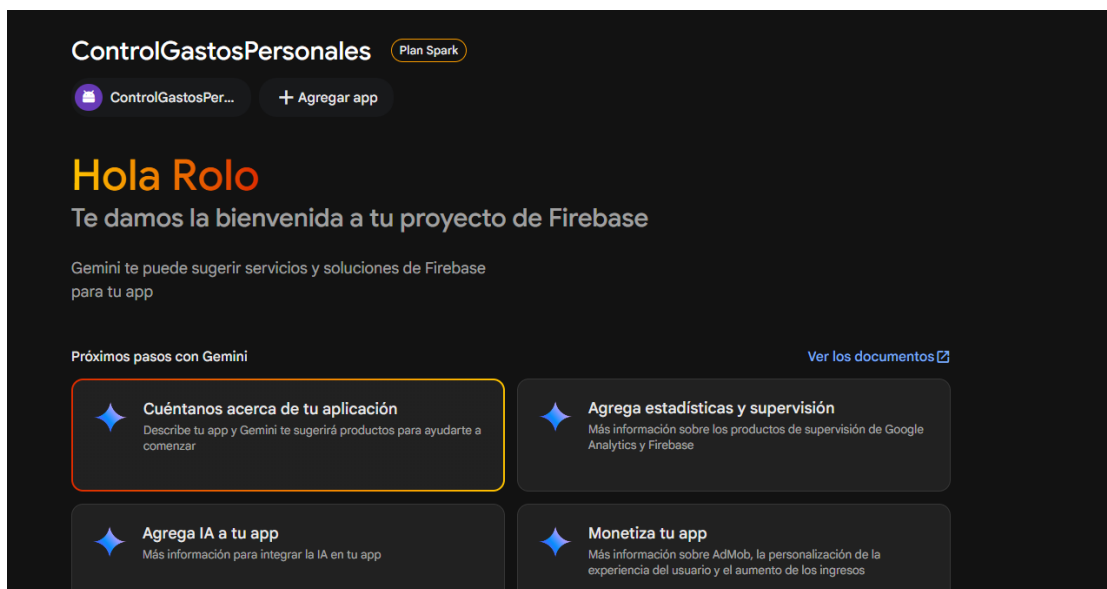
3.3. Creación del proyecto en Firebase Console

Luego se creó un proyecto en Firebase Console con el nombre ControlGastosPersonales. Este proyecto fue utilizado para configurar los servicios de autenticación y base de datos en la nube.

Para esta actividad no se activó Google Analytics, ya que los requisitos principales se enfocan en Firebase Authentication y Firebase Firestore.

Figura 3

Proyecto ControlGastosPersonales creado en Firebase Console.



Nota. Se muestra el proyecto creado en Firebase Console, desde donde se configuraron los servicios necesarios para la aplicación.

3.4. Registro de la aplicación Android en Firebase

Dentro del proyecto de Firebase se agregó una aplicación Android. Para vincular correctamente Android Studio con Firebase, se registró el nombre del paquete usado por la aplicación:

`com.example.controlgastos`

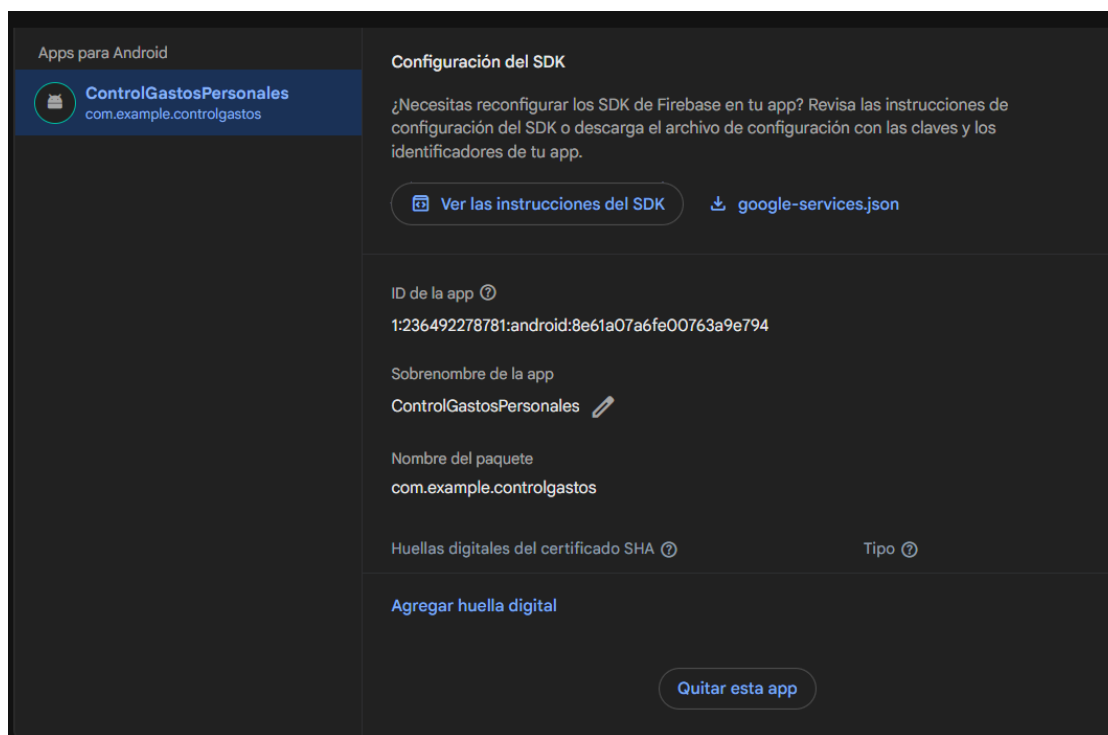
También se asignó como sobrenombre de la aplicación:

`ControlGastosPersonales`

Este paso permite que Firebase identifique la aplicación Android que utilizará sus servicios.

Figura 4

Registro de la aplicación Android en Firebase.



Nota. Se observa el registro de la app Android con el paquete com.example.controlgastos, necesario para vincular la aplicación con Firebase.

3.5. Integración del archivo google-services.json

Después de registrar la aplicación Android, Firebase generó el archivo de configuración llamado google-services.json. Este archivo contiene la configuración necesaria para conectar la aplicación Android con el proyecto de Firebase correspondiente.

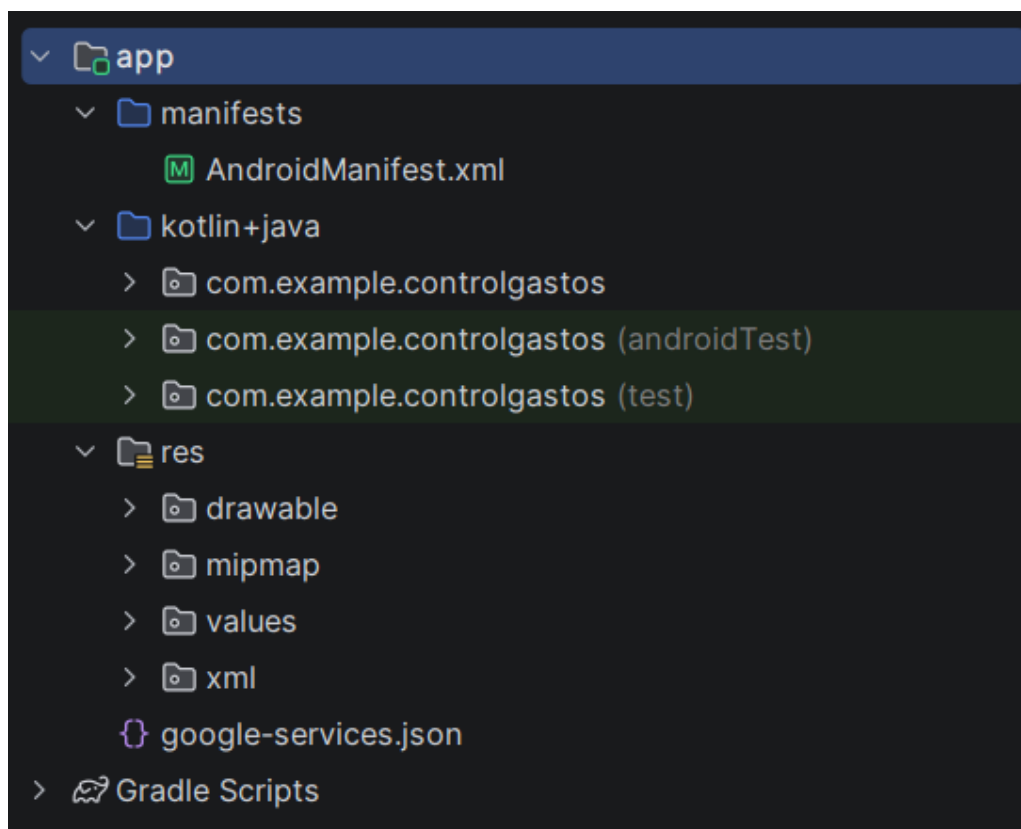
El archivo fue descargado desde Firebase Console y colocado dentro del módulo app del proyecto Android Studio, en la siguiente ubicación:

ControlGastosPersonales/app/google-services.json

Colocar este archivo en la ubicación correcta es necesario para que las dependencias de Firebase puedan reconocer el proyecto asociado.

Figura 5

Archivo google-services.json agregado al módulo app.



Nota. Se muestra el archivo google-services.json ubicado dentro del módulo app del proyecto Android.

3.6. Configuración del SDK de Firebase en Gradle

Para integrar Firebase en Android, se agregó el complemento de Google Services en los archivos Gradle del proyecto. Como el proyecto fue creado con Kotlin DSL, se utilizaron archivos build.gradle.kts (Firebase, s. f.-d).

En el archivo Gradle de nivel raíz se agregó el plugin de Google Services:

```
id("com.google.gms.google-services") version "4.4.4" apply false
```

En el archivo Gradle del módulo app se aplicó el plugin correspondiente y se agregaron las dependencias necesarias para Firebase Authentication y Firebase Firestore.

Figura 6

Configuración del plugin de Google Services en Gradle.

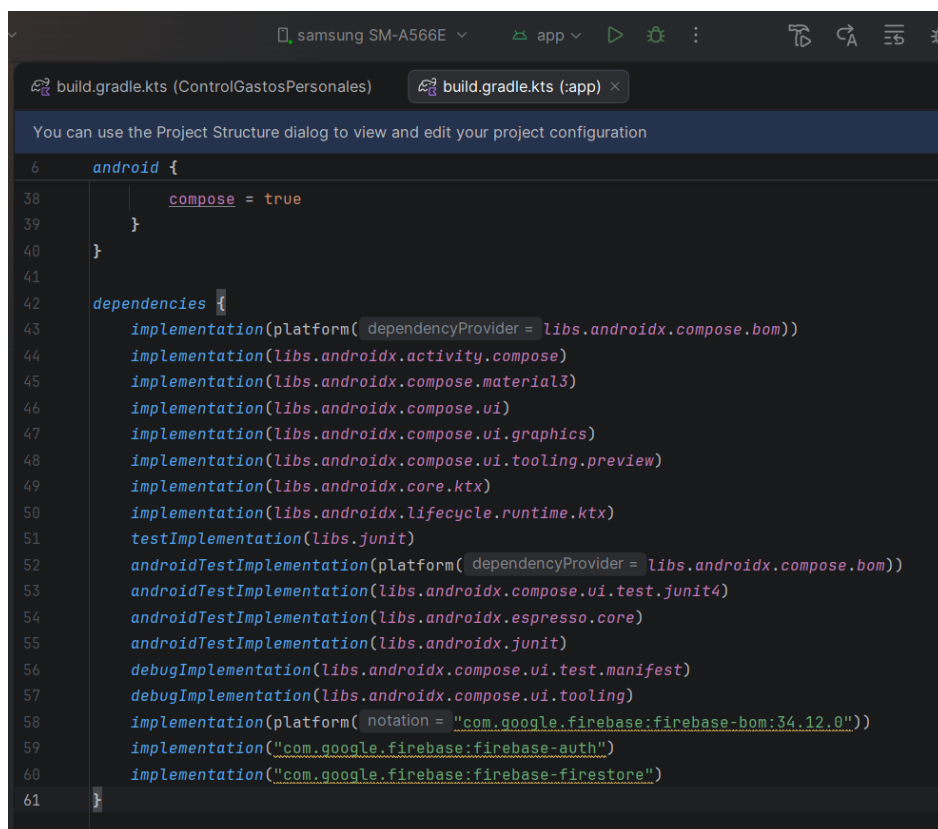


```
1  plugins {  
2      alias(libs.plugins.android.application)  
3      alias(libs.plugins.kotlin.compose)  
4      id("com.google.gms.google-services")  
5  }  
6  android {  
7      namespace = "com.example.controlgastos"  
8      compileSdk {  
9          version = release( version = 36) {  
10             minorApiLevel = 1  
11         }  
12     }  
13 }  
14 defaultConfig {  
15     applicationId = "com.example.controlgastos"
```

Nota. Se observa la configuración del plugin de Google Services en los archivos Gradle del proyecto.

Figura 7

Dependencias de Firebase Authentication y Firebase Firestore en Gradle.



```

6  android {
38      compose = true
39  }
40  }
41
42  dependencies {
43      implementation(platform(dependencyProvider = libs.androidx.compose.bom))
44      implementation(libs.androidx.activity.compose)
45      implementation(libs.androidx.compose.material3)
46      implementation(libs.androidx.compose.ui)
47      implementation(libs.androidx.compose.ui.graphics)
48      implementation(libs.androidx.compose.ui.tooling.preview)
49      implementation(libs.androidx.core.ktx)
50      implementation(libs.androidx.lifecycle.runtime.ktx)
51      testImplementation(libs.junit)
52      androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))
53      androidTestImplementation(libs.androidx.compose.ui.test.junit4)
54      androidTestImplementation(libs.androidx.espresso.core)
55      androidTestImplementation(libs.androidx.junit)
56      debugImplementation(libs.androidx.compose.ui.test.manifest)
57      debugImplementation(libs.androidx.compose.ui.tooling)
58      implementation(platform(notation = "com.google.firebase:firebase-bom:34.12.0"))
59      implementation("com.google.firebase:firebase-auth")
60      implementation("com.google.firebase:firebase-firestore")
61  }

```

Nota. Se muestran las dependencias necesarias para utilizar Firebase Authentication y Firebase Firestore en la aplicación Android.

3.7. Activación de Firebase Authentication

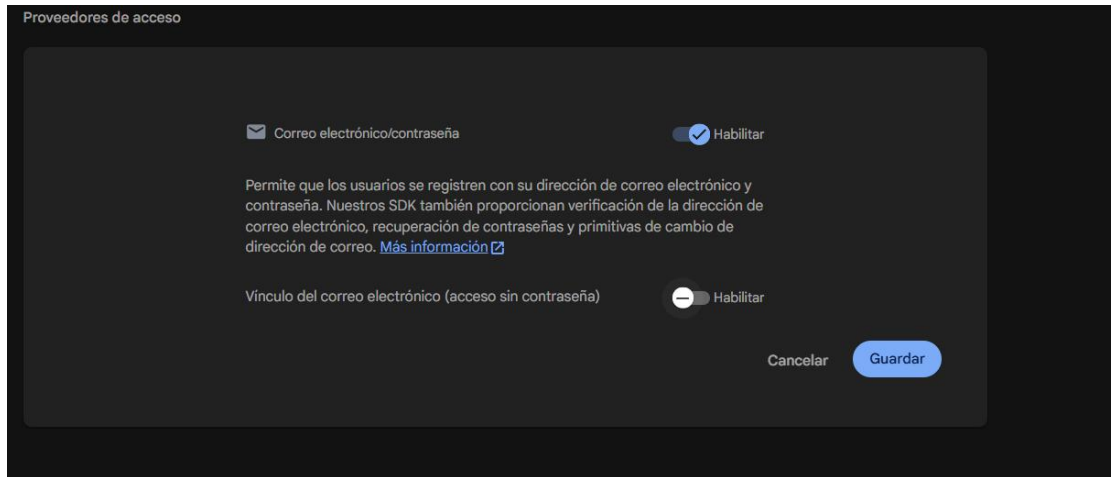
Después de integrar el SDK, se activaron los métodos de autenticación requeridos por la actividad. Firebase Authentication permite habilitar distintos proveedores de acceso en Android, incluyendo correo electrónico, contraseña y Google (Firebase, s. f.-a; Firebase, s. f.-d). Para este proyecto, los métodos principales fueron:

- Correo electrónico y contraseña.
- Google.

El método de correo electrónico permite que los usuarios se registren e inicien sesión con credenciales propias. El método de Google permite iniciar sesión mediante una cuenta de Google.

Figura 8

Configuración del proveedor de correo electrónico y contraseña.



Nota. Se observa la habilitación del proveedor de autenticación por correo electrónico y contraseña en Firebase Authentication.

Figura 9

Proceso de configuración del proveedor Google.

Para el proveedor (paso 2 de 2)

Google Habilitar

Importante: Para habilitar el Acceso con Google en tus apps para Android, debes proporcionar la [huella digital de lanzamiento SHA-1](#) para cada app (ve a [Configuración del proyecto > Tus apps](#)).

⚙️ Actualiza el siguiente [parámetro de configuración a nivel de proyecto](#) para continuar

Nombre público del proyecto ⓘ

ControlGastosPersonales

Correo electrónico de asistencia del proyecto ⓘ

akatsukimartir2@gmail.com

Agrega a la lista de entidades seguras los ID de cliente de proyectos externos (opcional) ⓘ ✓

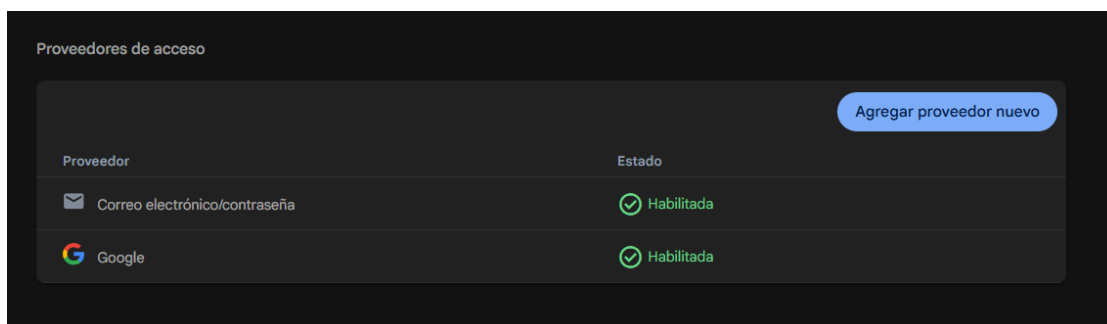
Configuración del SDK web ⓘ ✓

Cancelar Guardar

Nota. Se muestra el proceso de configuración del proveedor Google dentro de Firebase Authentication.

Figura 10

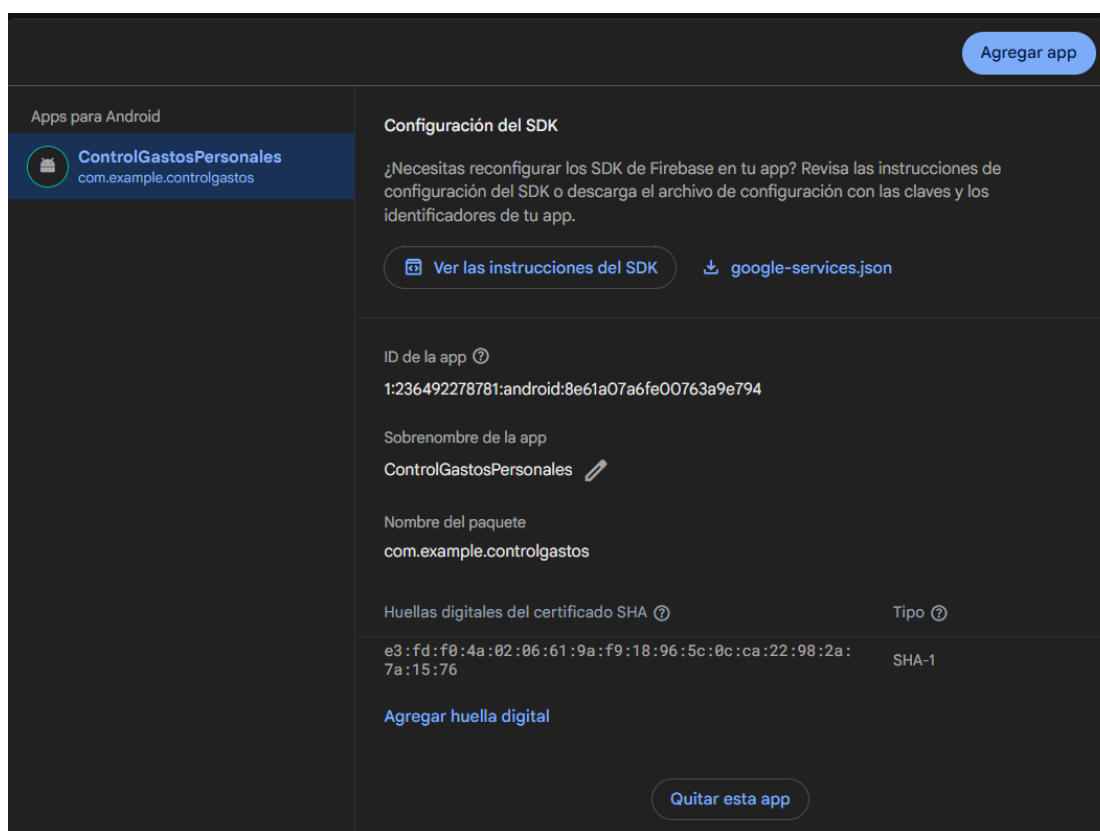
Métodos de autenticación activados en Firebase Authentication.



Nota. Se muestran habilitados los métodos de autenticación por correo electrónico/contraseña y Google.

Figura 11

Huella digital SHA-1 agregada a la aplicación Android en Firebase.



Nota. Se muestra la huella digital SHA-1 agregada a la app Android, necesaria para el inicio de sesión con Google.

3.8. Configuración de Firebase Firestore

Para almacenar los gastos personales se utilizó Firebase Firestore, una base de datos en la nube que permite crear una base de datos, agregar información y consultarla desde Firebase Console (Firebase, s. f.-b). Esta base de datos permitió guardar los gastos registrados por cada usuario autenticado.

La estructura utilizada para los datos fue la siguiente:

users

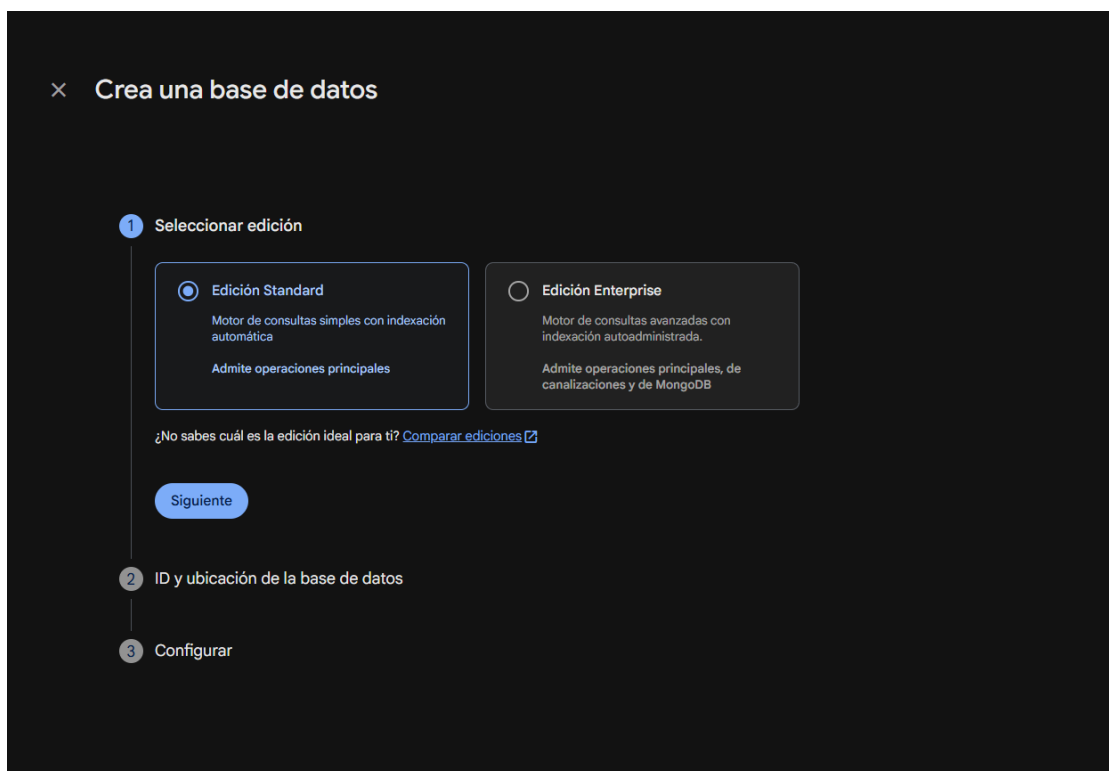
userId

expenses
expenseId
name
amount
category
date
month
createdAt
userId

Esta estructura permite que cada usuario tenga sus propios gastos en una subcolección independiente, evitando mezclar información entre diferentes cuentas.

Figura 12

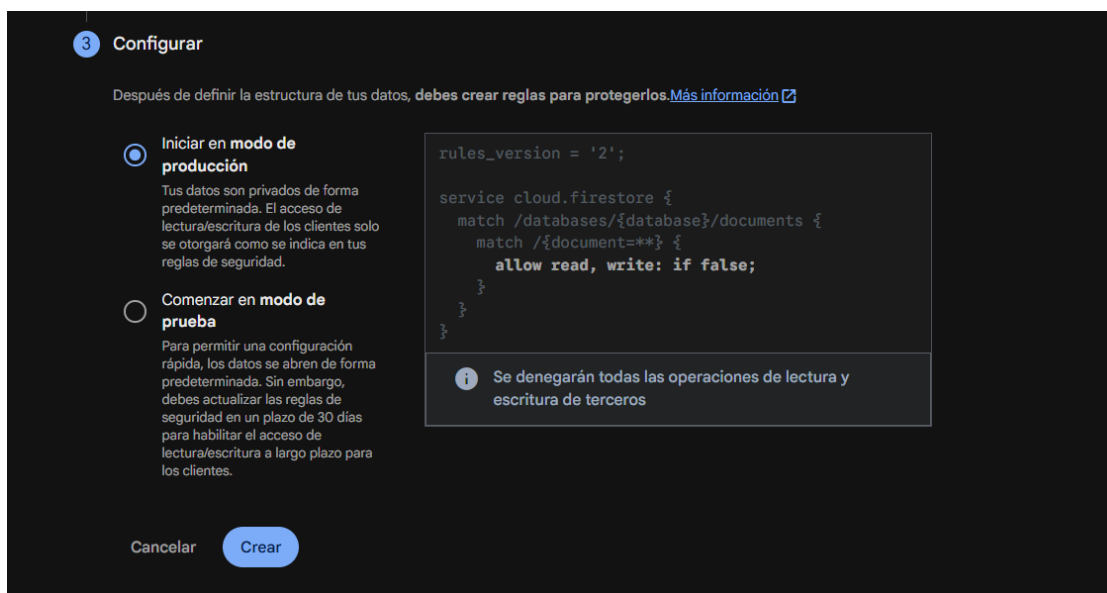
Selección de la edición Standard de Cloud Firestore.



Nota. Se observa la selección de la edición Standard para crear la base de datos Firestore.

Figura 13

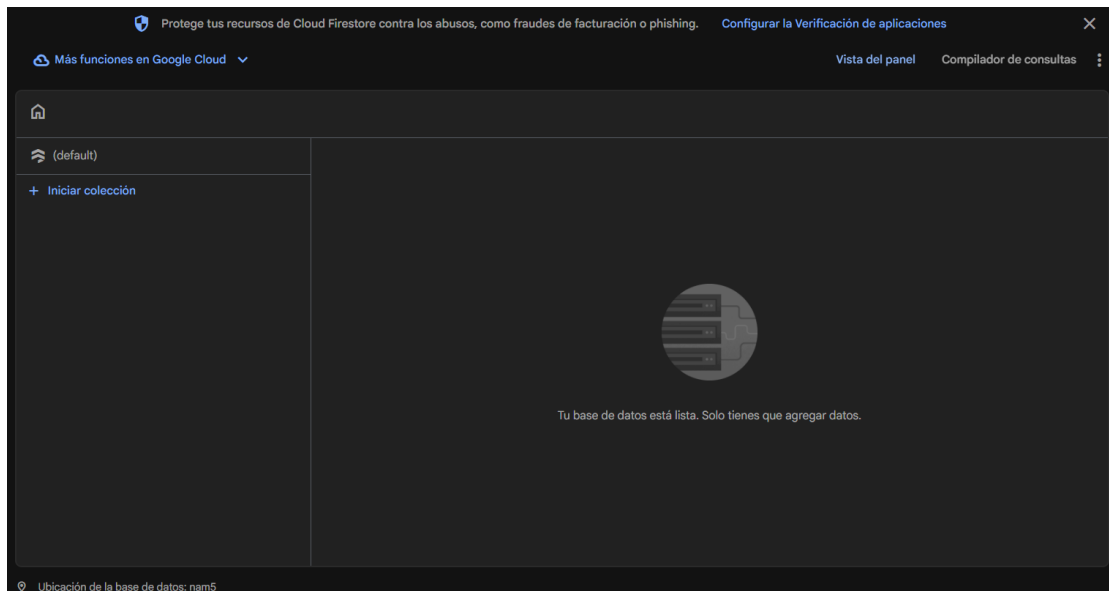
Configuración inicial de Firestore en modo de producción.



Nota. Se muestra la configuración inicial de reglas en modo de producción para proteger los datos.

Figura 14

Base de datos Firestore creada para la aplicación.



Nota. Se muestra la base de datos Cloud Firestore creada para almacenar los gastos personales de los usuarios autenticados.

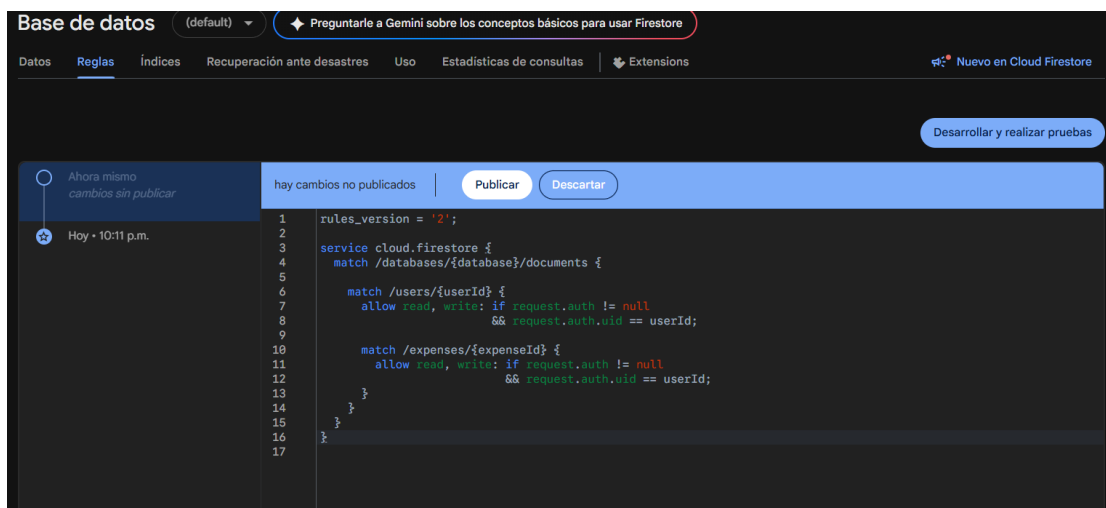
3.9. Reglas de seguridad de Firestore

Para proteger la información, Firestore se configuró mediante reglas de seguridad que permiten que cada usuario solo pueda leer y escribir sus propios gastos. La regla se basa en comparar el identificador del usuario autenticado con el identificador del usuario dentro de la ruta de la base de datos (Firebase, s. f.-c; Firebase, s. f.-e).

De esta manera, un usuario autenticado no podrá consultar ni modificar los gastos de otro usuario.

Figura 15

Reglas de seguridad de Firestore para proteger los gastos por usuario.



Nota. Se muestran las reglas de seguridad configuradas para permitir que cada usuario autenticado acceda únicamente a su propio documento y a sus propios gastos.

4. DESARROLLO DE LA APLICACIÓN DE CONTROL DE GASTOS

4.1. Diseño general de la aplicación

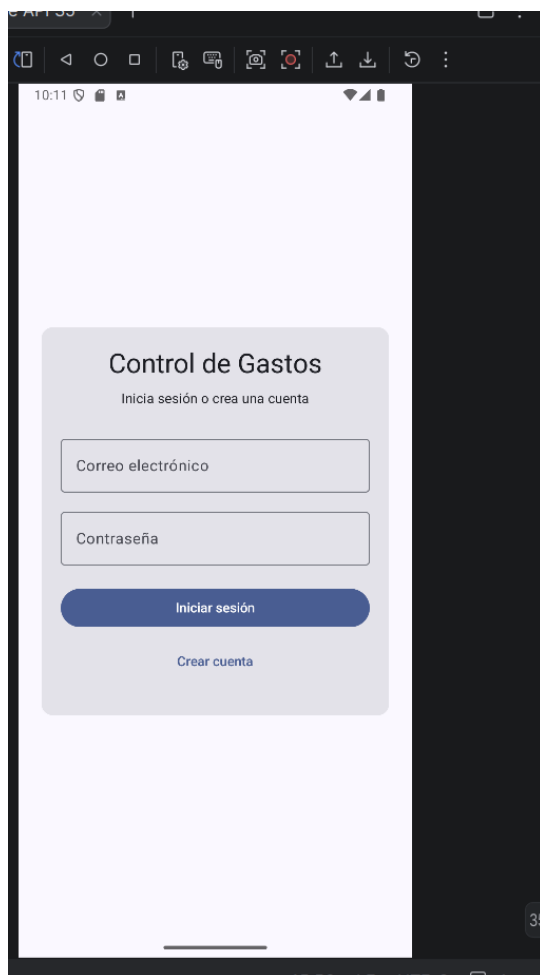
La aplicación de control de gastos personales se desarrolla en Android Studio utilizando Kotlin y Jetpack Compose. Su diseño inicial se organiza a partir de una pantalla de autenticación, desde la cual el usuario puede iniciar sesión o crear una cuenta mediante correo electrónico y contraseña.

Esta pantalla es el primer punto de acceso a la aplicación, ya que permite identificar al usuario antes de utilizar las funciones internas del sistema. La autenticación es necesaria para que los gastos registrados posteriormente puedan asociarse al usuario correspondiente y almacenarse de forma segura en Firebase Firestore.

El diseño de la pantalla se mantiene claro y funcional, incluyendo campos para correo electrónico y contraseña, un botón para iniciar sesión y una opción para crear una cuenta nueva. De esta manera, la aplicación cumple con la base necesaria para continuar con el registro de gastos, el historial y el cálculo del total mensual.

Figura 16

Pantalla de inicio de sesión de la aplicación.



Nota. En la figura se muestra la pantalla inicial de autenticación, donde el usuario puede iniciar sesión o crear una cuenta mediante correo electrónico y contraseña.

4.2. Registro e inicio de sesión con correo electrónico

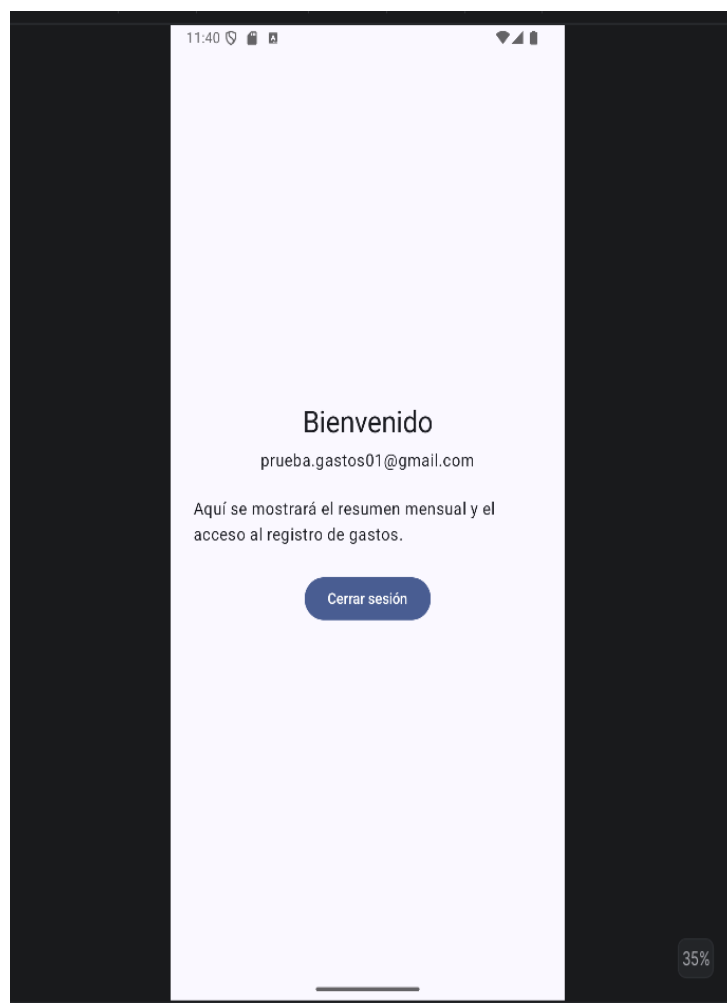
Después de implementar la pantalla inicial de autenticación, se realizó la prueba de registro e inicio de sesión mediante correo electrónico y contraseña. Para esta función se utilizó Firebase Authentication, permitiendo que el usuario cree una cuenta y acceda a la aplicación con sus credenciales.

La aplicación valida que el correo electrónico no esté vacío, que tenga un formato válido y que la contraseña cumpla una longitud mínima. Estas validaciones permiten reducir errores durante el registro y mejorar la experiencia del usuario.

Cuando el usuario se registra correctamente, Firebase Authentication almacena la cuenta en la consola del proyecto. Posteriormente, la aplicación reconoce al usuario autenticado y muestra una pantalla de bienvenida con el correo electrónico de la cuenta activa.

Figura 17

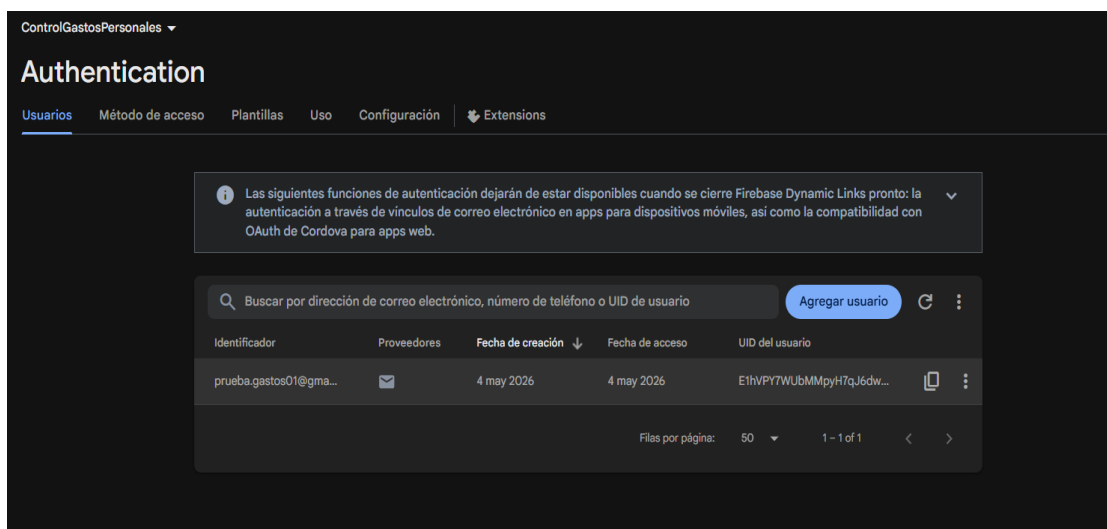
Inicio de sesión exitoso mediante correo electrónico.



Nota. En la figura se muestra la pantalla de bienvenida después de registrar o iniciar sesión correctamente mediante correo electrónico y contraseña.

Figura 18

Usuario registrado en Firebase Authentication.



Nota. En la figura se muestra el usuario creado dentro de Firebase Authentication, confirmando que el registro mediante correo electrónico fue realizado correctamente.

4.3. Registro de gastos

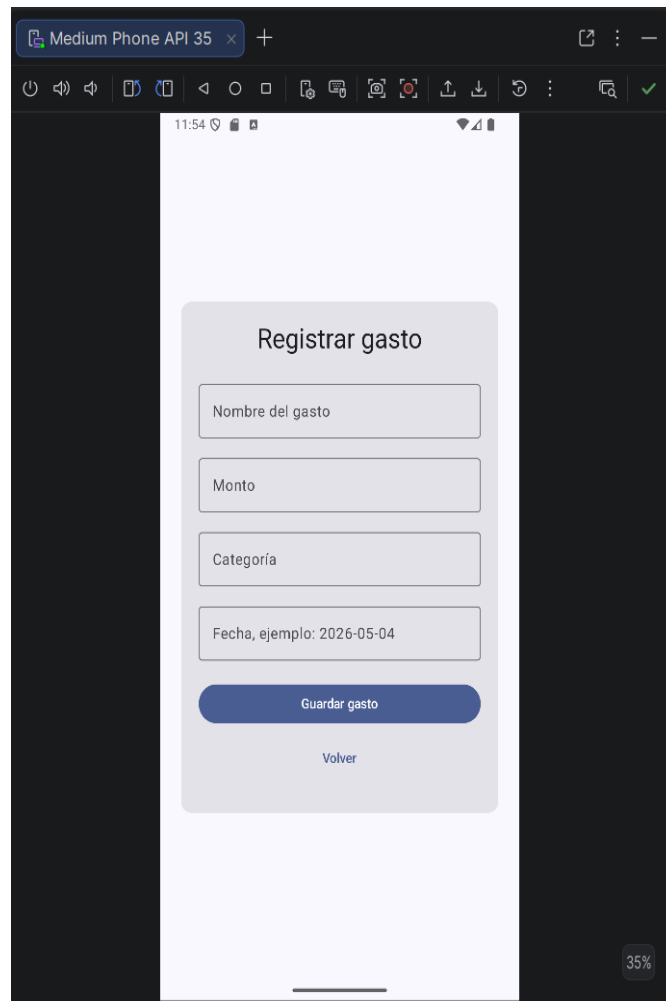
Después de implementar la autenticación, se agregó una pantalla para el registro de gastos personales. Esta pantalla permite que el usuario ingrese la información básica de cada gasto: nombre, monto, categoría y fecha.

El formulario incluye validaciones básicas para evitar registros incompletos o incorrectos. El nombre del gasto no puede estar vacío, el monto debe ser numérico y mayor que cero, la categoría debe completarse y la fecha debe ingresarse para identificar cuándo se realizó el gasto.

Esta pantalla representa una de las funciones principales de la aplicación, ya que permite capturar los datos que posteriormente serán almacenados en Firebase Firestore y asociados al usuario autenticado.

Figura 19

Pantalla de registro de gastos.



The image shows a mobile application interface for recording expenses. The form is titled "Registrar gasto" and contains four input fields: "Nombre del gasto", "Monto", "Categoría", and "Fecha, ejemplo: 2026-05-04". Below the fields are two buttons: "Guardar gasto" (Save expense) and "Volver" (Back). The form is displayed on a smartphone screen with a status bar at the top showing the time 11:54 and battery level 35%.

Nota. En la figura se muestra el formulario utilizado para registrar un gasto personal, incluyendo los campos de nombre, monto, categoría y fecha.

4.4. Almacenamiento de gastos en Firestore

Después de validar los datos ingresados en el formulario, la aplicación guarda cada gasto en Firebase Firestore. Para mantener la información organizada y protegida, los gastos se

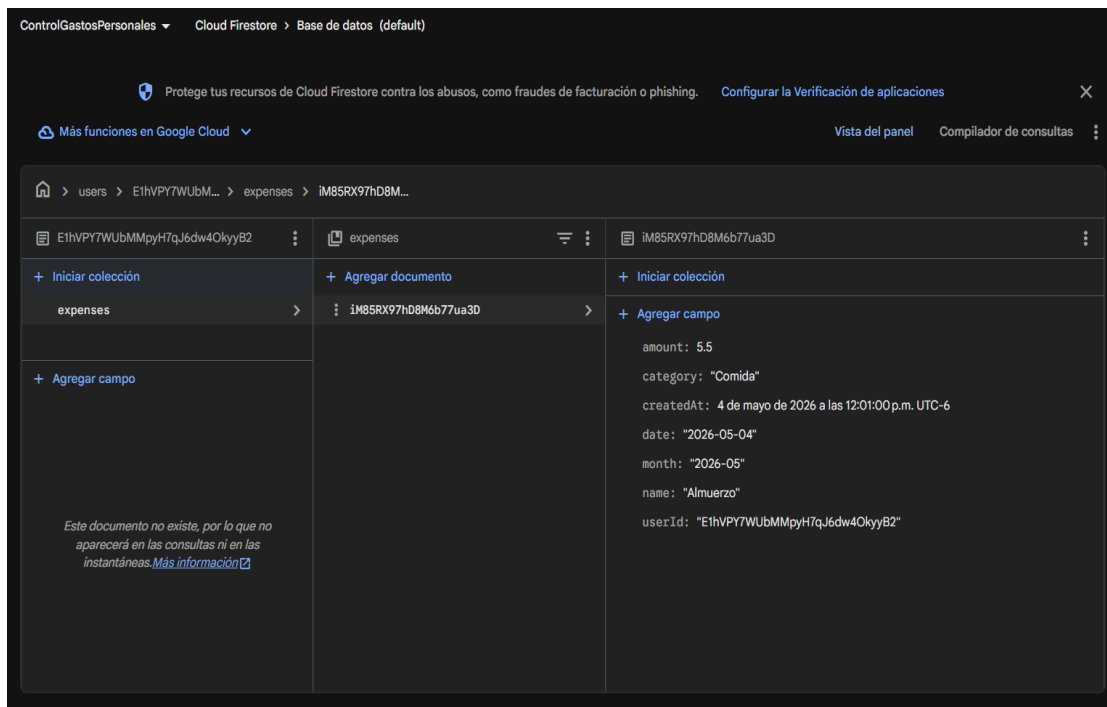
almacenan dentro de una colección principal llamada users, utilizando como documento el identificador del usuario autenticado.

Dentro de cada usuario se utiliza una subcolección llamada expenses, donde se registra cada gasto individual. Esta estructura permite que los gastos queden asociados al usuario que inició sesión y evita mezclar información entre diferentes cuentas.

Cada documento de gasto contiene los campos name, amount, category, date, month, createdAt y userId. Estos datos permiten mostrar posteriormente el historial de gastos, filtrar por mes y calcular el total mensual.

Figura 20

Gasto almacenado en Firebase Firestore.



Nota. En la figura se observa el gasto guardado en Firestore dentro de la subcolección expenses del usuario autenticado, incluyendo nombre, monto, categoría, fecha, mes, fecha de creación e identificador del usuario.

4.5. Visualización del historial de gastos

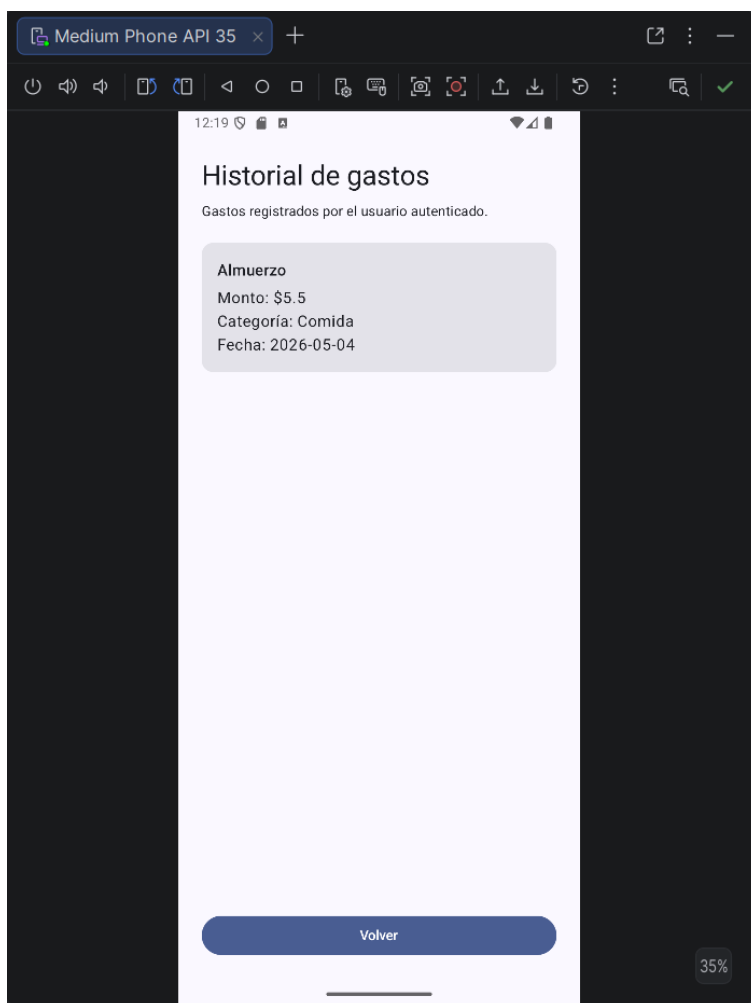
Después de guardar los gastos en Firebase Firestore, se implementó una pantalla para visualizar el historial de gastos registrados por el usuario autenticado. Esta pantalla consulta la subcolección expenses correspondiente al usuario actual y muestra los datos almacenados.

Cada elemento del historial presenta información básica del gasto, incluyendo el nombre, monto, categoría y fecha. Esto permite que el usuario revise los gastos ingresados y confirme que la información se está recuperando correctamente desde la base de datos.

La visualización del historial es una función importante dentro de la aplicación, ya que permite comprobar que los registros no solo se guardan en Firestore, sino que también pueden consultarse desde la interfaz de usuario.

Figura 21

Pantalla de historial de gastos.



Nota. En la figura se muestra el historial de gastos recuperado desde Firestore, donde aparece el gasto registrado con su nombre, monto, categoría y fecha.

4.6. Cálculo del total mensual

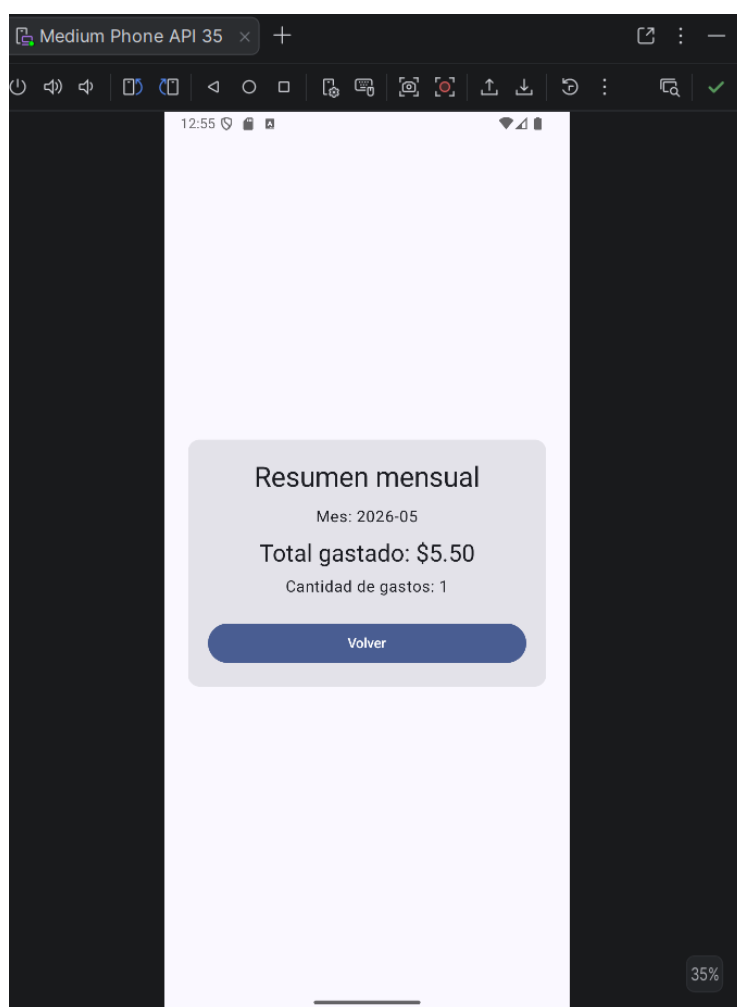
Para completar la funcionalidad de control de gastos, se implementó una pantalla de resumen mensual. Esta pantalla consulta los gastos almacenados en Firebase Firestore y filtra los registros correspondientes al mes actual.

El cálculo se realiza sumando los montos de los gastos cuyo campo month coincide con el mes seleccionado por la aplicación. Además del total gastado, también se muestra la cantidad de gastos registrados durante ese período.

Esta función permite que el usuario tenga una vista resumida de sus gastos mensuales, facilitando el análisis de su consumo y complementando el historial detallado de gastos.

Figura 22

Pantalla de resumen mensual.



Nota. En la figura se muestra el cálculo del total mensual de gastos, incluyendo el mes consultado, el total gastado y la cantidad de gastos registrados.

4.7. Validaciones y manejo de errores

La aplicación implementa validaciones de entrada y manejo de errores con el propósito de asegurar que la información ingresada por el usuario cumpla con los requisitos mínimos antes de ser procesada o almacenada. Estas validaciones contribuyen a mejorar la calidad de los datos y a brindar una experiencia de uso más clara y controlada.

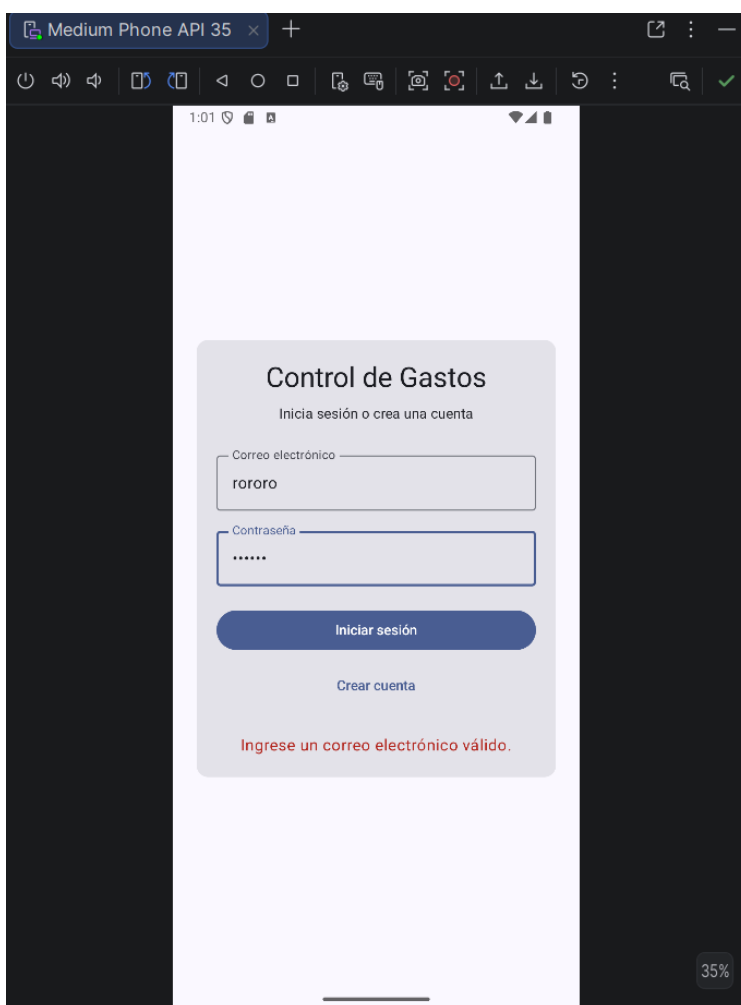
En la pantalla de inicio de sesión, se verifica que el correo electrónico no esté vacío y que tenga un formato válido. Si el usuario ingresa un correo incorrecto, la aplicación muestra un mensaje de advertencia y evita continuar con el proceso de autenticación hasta que el dato sea corregido.

De forma similar, en el formulario de registro de gastos se valida que todos los campos obligatorios estén completos y que el monto corresponda a un valor numérico válido. Si el usuario introduce texto no numérico en el campo del monto, la aplicación muestra un mensaje de error y evita guardar el registro.

Además de las validaciones locales, la aplicación también contempla el manejo de errores provenientes de Firebase Authentication y Cloud Firestore. Cuando ocurre una falla en la autenticación o en el almacenamiento de datos, el sistema informa al usuario mediante mensajes descriptivos, lo que facilita la identificación del problema y mejora la robustez general del sistema.

Figura 23

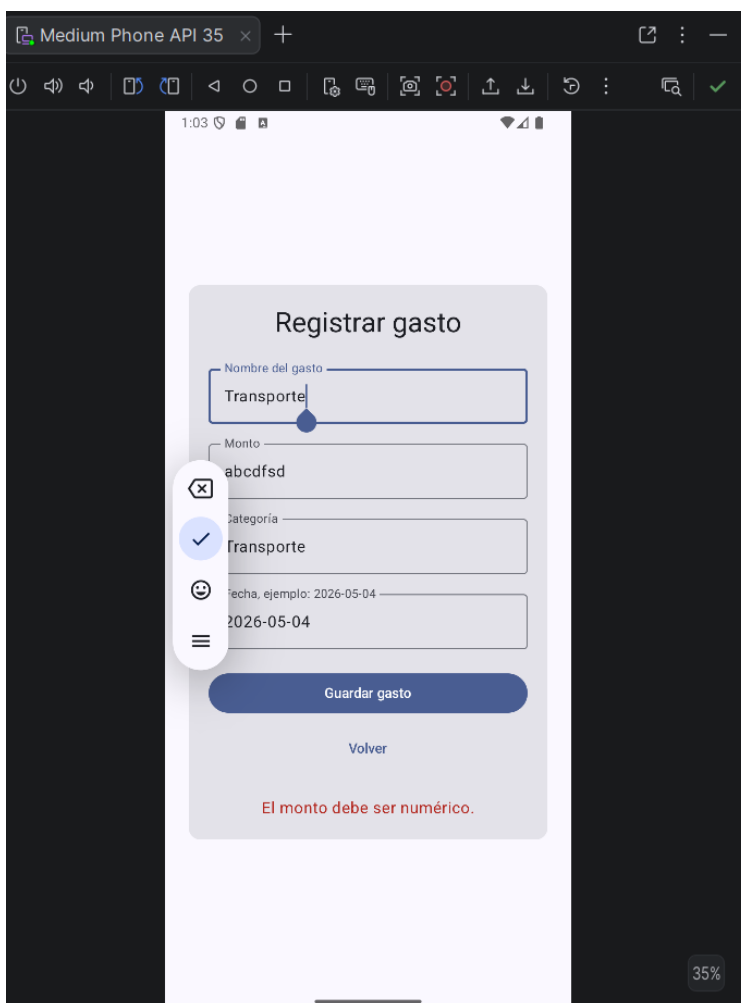
Validación de correo electrónico en la pantalla de inicio de sesión.



Nota. La figura muestra el mensaje de validación presentado cuando el usuario intenta iniciar sesión con un correo electrónico que no cumple con el formato esperado.

Figura 24

Validación del campo monto en el formulario de registro de gastos.



The screenshot shows a mobile application interface for registering an expense. The form is titled "Registrar gasto" and contains the following fields:

- Nombre del gasto:** A text input field containing the word "Transporte".
- Monto:** A text input field containing the string "abcbfsd".
- Categoría:** A text input field containing the word "Transporte".
- Fecha, ejemplo: 2026-05-04:** A date input field containing the date "2026-05-04".

Below the input fields are two buttons: "Guardar gasto" (a blue button) and "Volver" (a smaller, lighter button). A red error message, "El monto debe ser numérico.", is displayed at the bottom of the form area. A floating action menu is visible on the left side of the form, containing icons for cancel (X), confirm (checkmark), smiley face, and a hamburger menu.

Nota. La figura muestra el mensaje de error generado cuando el usuario ingresa un valor no numérico en el campo correspondiente al monto del gasto.

5. Capturas de pantalla del proyecto en ejecución

En esta sección se presentan las capturas finales de la aplicación ControlGastosPersonales en funcionamiento. Las imágenes muestran el flujo principal del sistema, incluyendo autenticación, dashboard, registro de gastos, selección de fecha, historial, edición, eliminación, resumen mensual y almacenamiento de datos en Firebase.

5.1. Pantalla de inicio de sesión

La pantalla de inicio de sesión permite al usuario ingresar con correo electrónico y contraseña, o utilizar la opción de autenticación con Google. Esta pantalla representa el punto de entrada principal de la aplicación y mantiene una interfaz visual coherente con el estilo moderno aplicado al proyecto.

Figura 25

Pantalla de inicio de sesión de la aplicación.



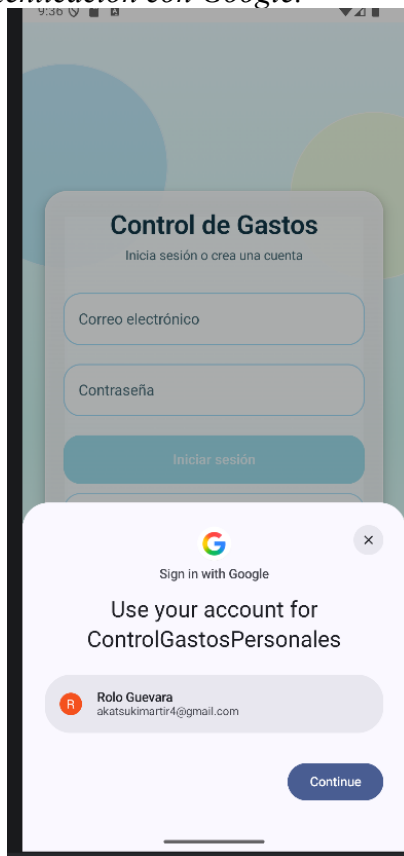
Nota. Captura final de la pantalla de inicio de sesión de la aplicación ControlGastosPersonales.

5.2. Inicio de sesión con Google

La aplicación incorpora autenticación con Google mediante Firebase Authentication. Esta funcionalidad permite que el usuario seleccione una cuenta de Google disponible en el dispositivo y acceda al sistema sin utilizar manualmente una contraseña dentro de la aplicación.

Figura 26

Selección de cuenta para autenticación con Google.



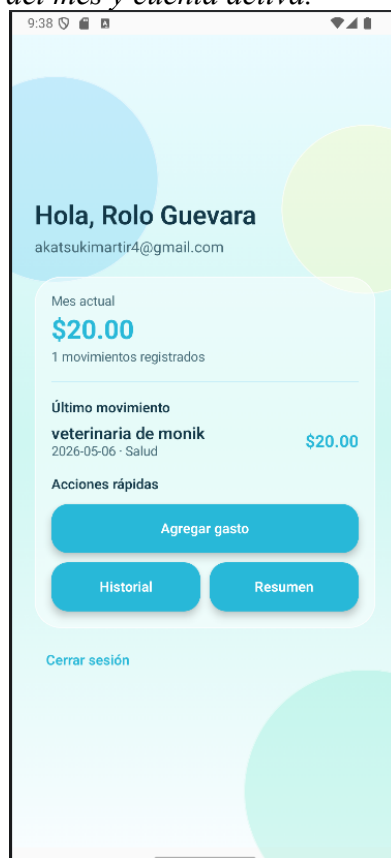
Nota. Captura final del selector de cuenta utilizado para iniciar sesión con Google.

5.3. Dashboard principal

Después de iniciar sesión, el usuario accede al dashboard principal. Esta pantalla muestra un saludo personalizado, el correo completo de la cuenta activa, el total gastado durante el mes actual, la cantidad de movimientos registrados y el último movimiento realizado. También incluye accesos rápidos para agregar gastos, consultar el historial y revisar el resumen mensual.

Figura 27

Panel principal con resumen del mes y cuenta activa.



Nota. Captura final del dashboard principal con el resumen mensual de la cuenta activa.

5.4. Registro de gastos

La pantalla de registro permite ingresar un nuevo gasto indicando nombre, monto, categoría y fecha. Las categorías se muestran como opciones seleccionables para evitar errores de escritura y mejorar la experiencia del usuario.

Figura 28

Pantalla para registrar un nuevo gasto.

The screenshot shows a mobile application interface for recording an expense. The title is "Registrar gasto". Below the title is a subtitle "Guarda nombre, monto, categoría y fecha". The form has three main sections: "Nombre del gasto" with a text input field containing "Internet"; "Monto" with a currency selector (a green circle with a white "\$" symbol) and a text input field containing "25"; and "Categoría" with a grid of buttons: "Comida", "Transporte", "Estudio", "Servicios" (which is highlighted in blue), "Salud", "Otros", and "Entretenimiento". Below the category buttons is a "Fecha" section with a text input field containing "2026-05-05", a "Formato guardado: yyyy-MM-dd" label, and two links: "Elegir fecha" and "Usar fecha actual". At the bottom of the form are two buttons: "Guardar gasto" (highlighted in blue) and "Volver".

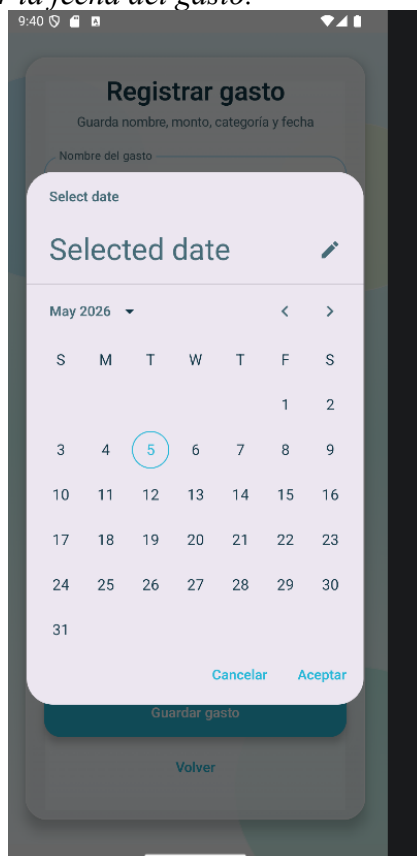
Nota. Captura final del formulario utilizado para registrar un nuevo gasto personal.

5.5. Selector visual de fecha

Para evitar que el usuario escriba manualmente la fecha, se implementó un selector visual tipo calendario. Esta mejora permite seleccionar la fecha de manera más clara y profesional, manteniendo internamente el formato necesario para el almacenamiento y los cálculos mensuales.

Figura 29

Calendario para seleccionar la fecha del gasto.



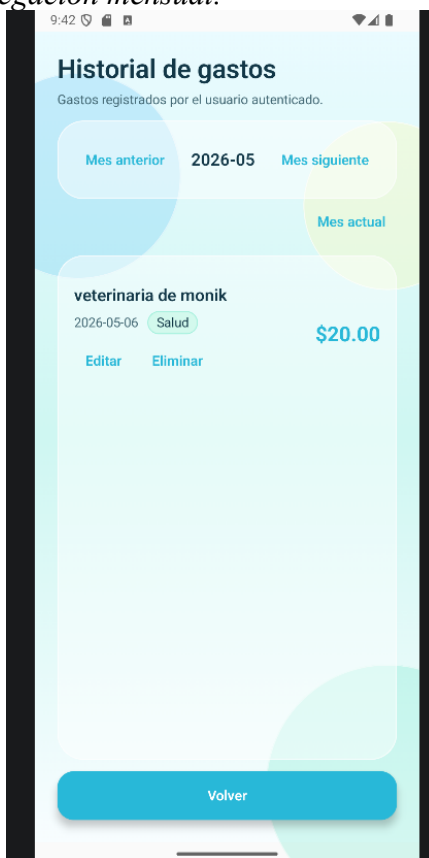
Nota. Captura final del selector visual de fecha implementado en la aplicación.

5.6. Historial mensual de gastos

El historial muestra los gastos registrados en una vista tipo ledger o journal, evitando el uso de tarjetas genéricas. Cada movimiento presenta el nombre del gasto, la fecha, la categoría, el monto y las acciones disponibles. Además, se incorporó navegación mensual para consultar gastos de diferentes meses.

Figura 30

Historial de gastos con navegación mensual.



Nota. Captura final del historial mensual de gastos con navegación por mes.

5.7. Edición de gastos

La aplicación permite editar gastos ya registrados desde el historial. Al seleccionar la opción de edición, se muestra un diálogo con los datos actuales del gasto, permitiendo modificar nombre, monto, categoría y fecha. Al guardar los cambios, la información se actualiza en Firestore.

Figura 31

Diálogo para editar un gasto registrado.

The screenshot shows a mobile application interface. At the top, the status bar displays the time 9:47 and various icons. The main screen is titled 'Historial de gastos' with a subtitle 'Gastos registrados por el usuario autenticado.' A modal dialog box titled 'Editar gasto' is centered on the screen. It contains the following fields and options: a text input for the description 'veterinaria de monik'; a currency icon (\$) and a numeric input for the amount '20.00'; a section titled 'Categoría' with buttons for 'Comida', 'Transporte', 'Estudio', 'Servicios', 'Salud' (which is highlighted in blue), 'Otros', and 'Entretenimiento'; and a date input field showing '2026-05-06'. At the bottom of the dialog are 'Cancelar' and 'Guardar' buttons. Below the dialog, on the main screen, is a 'Volver' button.

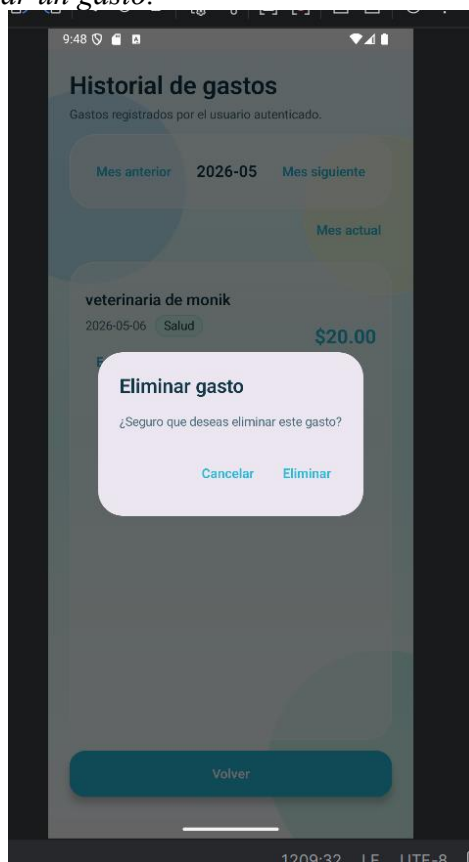
Nota. Captura final del diálogo utilizado para editar un gasto registrado.

5.8. Eliminación de gastos

El sistema también permite eliminar gastos registrados. Para evitar eliminaciones accidentales, se muestra un diálogo de confirmación antes de borrar el documento correspondiente en Firestore.

Figura 32

Confirmación para eliminar un gasto.



Nota. Captura final del diálogo de confirmación antes de eliminar un gasto.

5.9. Resumen mensual

La pantalla de resumen mensual presenta el total gastado y la cantidad de movimientos registrados en el mes seleccionado. Además, permite navegar entre meses, lo cual facilita revisar información anterior sin modificar la estructura de datos almacenada.

Figura 33

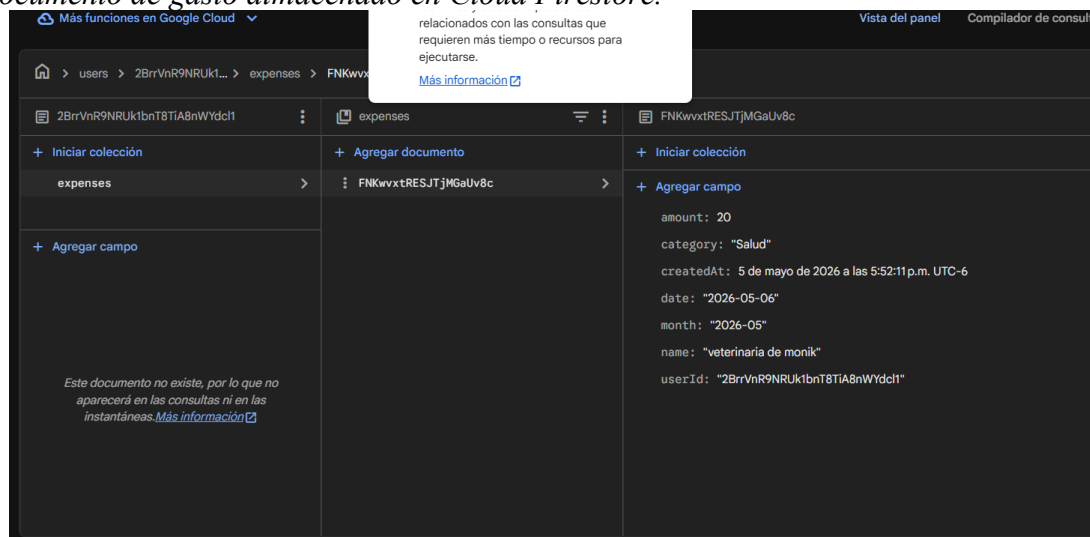
Resumen mensual con total y movimientos registrados.



Nota. Captura final del resumen mensual con total gastado y movimientos registrados.

5.10. Almacenamiento en Firestore

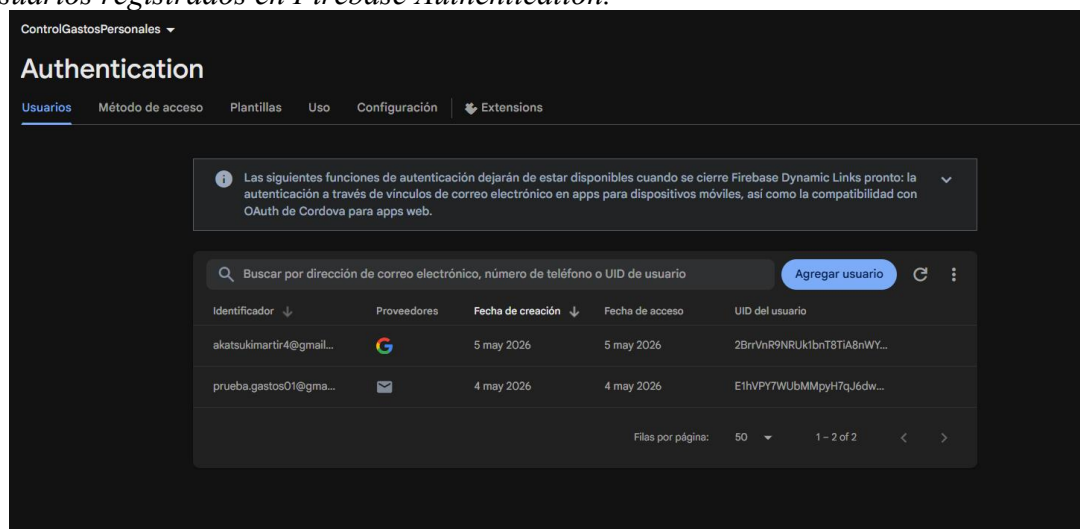
Los gastos registrados por el usuario se almacenan en Cloud Firestore bajo la estructura `users/{uid}/expenses/{expenseId}`. Cada documento conserva campos como nombre, monto, categoría, fecha, mes, fecha de creación e identificador del usuario, permitiendo mantener la información organizada por cuenta autenticada.

Figura 34*Documento de gasto almacenado en Cloud Firestore.*

Nota. Captura final del documento de gasto almacenado en Cloud Firestore.

5.11. Usuarios autenticados en Firebase

Firebase Authentication registra los usuarios que acceden a la aplicación. En la consola se puede observar tanto el usuario autenticado mediante Google como el usuario creado con correo electrónico y contraseña, validando que ambos métodos de autenticación funcionan correctamente.

Figura 35*Usuarios registrados en Firebase Authentication.*

Nota. Captura final de los usuarios registrados en Firebase Authentication.

6. Pruebas realizadas

Para verificar el correcto funcionamiento de la aplicación ControlGastosPersonales, se realizaron pruebas sobre los módulos principales del sistema. Estas pruebas permitieron comprobar el inicio de sesión, el registro de gastos, la consulta del historial, la edición y eliminación de registros, el resumen mensual y la conexión con Firebase Authentication y Cloud Firestore.

Las pruebas se realizaron utilizando el emulador de Android Studio y la consola de Firebase, validando tanto el comportamiento visual de la aplicación como el almacenamiento real de la información en la nube. La matriz siguiente organiza los resultados por módulo evaluado para facilitar la revisión técnica del proyecto.

Tabla 1

Matriz resumida de pruebas funcionales de la aplicación ControlGastosPersonales

N.º	Prueba realizada	Resultado obtenido
1	Inicio de sesión con correo electrónico	El usuario ingresó correctamente al dashboard principal utilizando credenciales válidas.
2	Inicio de sesión con Google	La aplicación permitió autenticarse con una cuenta de Google y acceder correctamente al sistema.
3	Registro de gasto	El gasto se guardó correctamente en Firestore y se mostró en el historial del usuario.
4	Validación de campos	El sistema impidió guardar registros incompletos o datos inválidos y mostró mensajes de validación.
5	Selector visual de fecha	La fecha se seleccionó mediante calendario y se almacenó correctamente con formato yyyy-MM-dd.
6	Historial, edición y eliminación	Los gastos se visualizaron por mes y pudieron editarse o eliminarse desde el historial sin errores.
7	Resumen mensual	El sistema mostró correctamente el total gastado y la cantidad de movimientos del mes seleccionado.
8	Almacenamiento y separación de datos en Firestore	Los gastos se almacenaron en users/{uid}/expenses y cada usuario gestionó únicamente sus propios registros.

Nota. Las pruebas fueron realizadas en el emulador de Android Studio y verificadas con Firebase Authentication y Cloud Firestore. Todas las pruebas registradas en la matriz fueron aprobadas, ya que el resultado obtenido coincidió con el comportamiento esperado.

En general, las pruebas realizadas demostraron que la aplicación cumple con las funciones principales requeridas. El sistema permite autenticar usuarios, registrar gastos, consultar información mensual, editar y eliminar movimientos, así como almacenar los datos en Firebase de forma organizada por usuario autenticado. Además, las validaciones implementadas ayudan a evitar registros incompletos o incorrectos, mejorando la confiabilidad de la aplicación.

7. Conclusiones personales del grupo

Como grupo, consideramos que este proyecto nos permitió comprender de forma más práctica cómo se construye una aplicación móvil conectada a servicios en la nube. Al inicio, el objetivo parecía centrarse únicamente en crear un sistema de inicio de sesión y registro de gastos, pero durante el desarrollo fuimos entendiendo que una aplicación funcional necesita integrar varias partes de manera ordenada: autenticación, almacenamiento de datos, validaciones, diseño de interfaz y pruebas de funcionamiento.

Uno de los aprendizajes más importantes fue comprender la relación entre Firebase Authentication y Cloud Firestore. La autenticación no solo sirve para iniciar sesión, sino también para identificar a cada usuario y separar correctamente su información. Gracias a esto, los gastos registrados se almacenan bajo el usuario autenticado, evitando que los datos se mezclen entre diferentes cuentas. Esta parte nos ayudó a entender mejor la importancia de organizar bien la estructura de la base de datos desde el inicio.

También aprendimos que la experiencia del usuario es una parte importante del desarrollo. Al principio, algunas funciones podían resolverse de forma básica, como escribir manualmente una fecha o mostrar pantallas simples. Sin embargo, al mejorar la aplicación con un selector visual de fecha, un dashboard más claro, navegación mensual, edición y eliminación de gastos, notamos que una interfaz más cuidada facilita el uso del sistema y reduce posibles errores.

Durante el proceso también enfrentamos dificultades técnicas, especialmente relacionadas con la configuración de Firebase, el uso de Google como método de autenticación y las consultas a Firestore. Estas situaciones nos ayudaron a analizar los errores con más criterio, buscar soluciones seguras y evitar cambios que pudieran romper funciones ya implementadas.

Por ejemplo, cuando Firestore solicitó un índice compuesto para una consulta, se decidió ajustar la lógica para ordenar los datos localmente, manteniendo la estructura de la base de datos sin complicar innecesariamente el proyecto.

En general, consideramos que la aplicación cumple con los objetivos principales de la actividad. El sistema permite iniciar sesión con correo electrónico y con Google, registrar gastos, seleccionar fechas de forma visual, consultar el historial, editar y eliminar movimientos, revisar el resumen mensual y verificar el almacenamiento de datos en Firebase. Más allá de cumplir los requisitos técnicos, el proyecto nos permitió practicar una forma de trabajo más ordenada, probando cada avance antes de continuar y respaldando los cambios importantes.

Como conclusión final, este proyecto nos dejó una mejor comprensión del desarrollo de aplicaciones móviles con Kotlin, Jetpack Compose y Firebase. También nos permitió valorar la importancia de las validaciones, la seguridad de los datos, la organización del código y la presentación final del sistema. Aunque todavía podrían agregarse mejoras en el futuro, como reportes más avanzados o filtros más específicos, la versión desarrollada representa una base funcional, clara y útil para el control de gastos personales.

8. Referencias

Android Developers. (s. f.-a). *Develop for Android*. <https://developer.android.com/develop>

Android Developers. (s. f.-b). *Get started with Jetpack Compose*.

<https://developer.android.com/develop/ui/compose/documentation>

Firebase. (s. f.-a). *Authenticate with Google on Android*.

<https://firebase.google.com/docs/auth/android/google-signin>

Firebase. (s. f.-b). *Get started with Cloud Firestore*.

<https://firebase.google.com/docs/firestore/quickstart>

Firebase. (s. f.-c). *Get started with Cloud Firestore Security Rules*.

<https://firebase.google.com/docs/firestore/security/get-started>

Firebase. (s. f.-d). *Get started with Firebase Authentication on Android*.

<https://firebase.google.com/docs/auth/android/start>

Firebase. (s. f.-e). *Structuring Cloud Firestore Security Rules*.

<https://firebase.google.com/docs/firestore/security/rules-structure>

Kotlin. (s. f.). *Kotlin for Android*. <https://kotlinlang.org/docs/android-overview.html>

9. Anexos

Anexo A. Aclaración sobre la evolución visual y funcional de la aplicación

Durante el desarrollo del proyecto, la aplicación ControlGastosPersonales pasó por una primera etapa mínima funcional y posteriormente por una etapa final de mejora visual y funcional. Las capturas incluidas en las primeras secciones del informe corresponden al proceso inicial de implementación, donde se comprobó el funcionamiento básico de la autenticación, el registro de gastos, el historial, el resumen mensual y la conexión con Firebase.

Estas primeras capturas se conservaron porque forman parte de la documentación del desarrollo paso a paso solicitado en la actividad. Su propósito es evidenciar cómo se configuraron e implementaron las funciones principales antes de aplicar los ajustes finales de interfaz y experiencia de usuario.

Por otro lado, las capturas presentadas en la sección 5 corresponden al acabado final de la aplicación. En ellas se muestra la versión más completa del sistema, incluyendo el dashboard principal mejorado, el inicio de sesión con Google, el selector visual de fecha, la navegación mensual en historial y resumen, la edición y eliminación de gastos, así como la evidencia final de Firebase Authentication y Cloud Firestore.

De esta manera, la diferencia entre las capturas iniciales y las capturas finales no representa una inconsistencia, sino la evolución natural del proyecto. Las primeras imágenes documentan el avance técnico de la implementación y las capturas de la sección 5 muestran el resultado final preparado para la entrega.

Anexo B. Enlace del video de presentación

Video de presentación: <https://youtu.be/oDUfcvOkAEU>

Repositorio en GitHub: <https://github.com/RolandoGio/Foro2ControGastosPersonales>